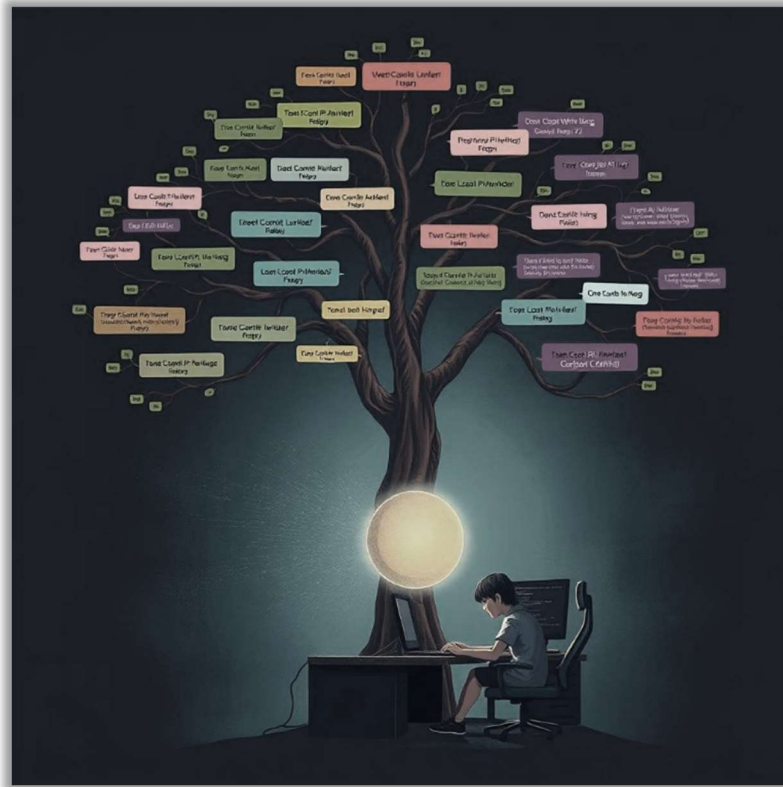


Capítulo 7

Herança



Fonte: Gamma – IA | Adaptada

7.1 Definição e Aplicação



A herança é um mecanismo que permite que uma classe (*subclasse*) reutilize os atributos e métodos de outra classe (*superclasse*). Esse conceito promove a reutilização de código e facilita a criação de hierarquias de classes, organizando melhor o sistema.

Imagine um mundo em que, para criar cada novo tipo de objeto, você precisasse reescrever todo o código do zero. Isso seria extremamente trabalhoso e ineficiente. A herança surge justamente como uma solução para esse problema, permitindo que classes compartilhem atributos e métodos de maneira organizada e coerente. Com a herança, é possível criar hierarquias de classes, tornando o código mais reutilizável, fácil de manter e de estender.

Neste capítulo, mergulharemos no conceito de herança na Orientação a Objetos. Você entenderá como esse mecanismo permite que *subclasses* herdem características de *superclasses*, como implementá-lo em Java e como representá-lo em diagramas UML. Além disso, exploraremos os benefícios, os cuidados necessários e as melhores práticas para utilizar a herança de forma eficiente e clara.

7.1.1 Metáfora do Mundo Real

Considere o conceito de “Veículo”. Essa é a *superclasse*, representando características comuns como: ter rodas, ser capaz de mover-se, ter um motor (no caso de veículos motorizados). Carros, motos, caminhões ou bicicletas seriam *subclasses* de Veículo, cada um adicionando suas próprias características (quantidade de rodas, tipo de motor ou se não tem motor, capacidade de carga, etc.).

7.1.2 Benefícios da Herança

- **Reutilização de Código:** Evita a duplicação de atributos e métodos comuns a várias classes.
- **Organização e Legibilidade:** Agrupa características genéricas em classes mais gerais (*superclasses*), mantendo o código mais limpo e claro.

- **Extensibilidade:** Facilita a adição de novas funcionalidades, criando novas *subclasses* sem alterar classes já existentes.
- **Especialização:** Permite criar versões mais específicas de uma classe geral, adaptando-a a contextos particulares.

7.2 Implementação de Herança em Java



A implementação do conceito de *Herança* em Java é feita utilizando a palavra-chave `extends`. Quando uma classe herda de outra, ela ganha acesso aos atributos e métodos da classe base (também chamada de superclasse), permitindo a reutilização de código e a especialização de comportamentos. Essa característica é especialmente útil para evitar a duplicação de código e para organizar melhor a lógica de um sistema.

Por exemplo, considere a superclasse **Animal**, que possui atributo `nome`, e um método chamado `emitirSom()`. Uma classe **Cachorro** pode herdar de **Animal** e adicionar comportamentos específicos, como o método `latir()`. A implementação ficaria assim:

Exemplo Simples:

Código_7.1: Classe **AnimalHeranca** em Java.

```
1 1 inheritor
1 public class AnimalHeranca {
2     private String nome;
3     public AnimalHeranca(String nome){
4         this.nome = nome;
5     }
6 1 override
6 public void emitirSom(){
7     System.out.println("Som genérico de animal.");
8 }
9 public String getNome(){
10     return nome;
11 }
12 }
```

Código_7.2: Classe **CachorroHeranca** em Java.

```

1  // Subclasse (classe filha)
2  public class CachorroHeranca extends AnimalHeranca {
3      public CachorroHeranca(String nome){
4          super(nome); /* Chama o construtor da superclasse
5                          para inicializar o atributo nome. */
6      }
7      // Sobrescrevendo o método emitirSom da superclasse.
8      @Override
9      public void emitirSom(){
10         System.out.println("Au, au!");
11     }
12 }

```

No Código_7.2, o termo '@Override', linha 8.

Código_7.3: Classe **TestaHeranca** em Java.

```

1  public class TestaHeranca {
2      public static void main(String[] args) {
3          HerancaAnimal animal = new HerancaAnimal( nome: "Bicho");
4          HerancaCachorro cachorro = new HerancaCachorro( nome: "Rex");
5
6          animal.emitirSom(); // Som genérico de animal.
7          cachorro.emitirSom(); // Au, au!
8      }
9  }

```

Saída:

```

Som genérico de animal.
Au, au!

```

⁷ A anotação @Override é usada para indicar que o método *emitirSom()* na classe *Cachorro* está sobrescrevendo o método *emitirSom()* na classe *Animal*. Isso evita erros de digitação no nome do método ou na lista de parâmetros, e não criando um novo método em vez de sobrescrever o existente.

No exemplo acima:

- **AnimalHeranca** é a *superclasse*.
- **CachorroHeranca** é a *subclasse* que herda atributos e métodos de **AnimalHeranca**, mas sobrescreve o método *emitirSom()* para fornecer um comportamento específico.

7.2.1 Chamada ao Construtor da Superclasse

Quando criamos uma *subclasse*, muitas vezes precisamos inicializar os atributos herdados a partir da *superclasse*. Para isso, usamos a palavra-chave *super()* no construtor da *subclasse*, chamando assim o construtor da *superclasse*.

Exemplo:

Código_7.4: Classe **GatoHeranca** em Java.

```
1 public class GatoHeranca extends AnimalHeranca {
2     public GatoHeranca(String nome) {
3         super(nome); // Chama o construtor de Animal
4     }
5
6     @Override
7     public void emitirSom() {
8         System.out.println("Miau!");
9     }
10 }
```

7.2.2 Herdando Atributos e Métodos

A *subclasse* herda todos os atributos e métodos públicos e protegidos (protected) da *superclasse*. Os elementos privados não são acessíveis diretamente, mas podem ser manipulados por meio de métodos públicos da *superclasse*.

7.2.3 Sobrescrita de Métodos

A herança, aliada ao polimorfismo, permite que um objeto de uma *subclasse* seja tratado como se fosse da *superclasse*, tornando o código mais flexível. Por exemplo, um *Animal* pode, em tempo de execução, ser um *Cachorro* ou um *Gato*, desde que siga o contrato⁸ da *superclasse*. Os Códigos_7.1, 7.2 e 7.3, implementam esses conceitos em Java.

Polimorfismo e Herança

A sobrescrita de métodos permite que uma *subclasse* forneça uma implementação específica de um método definido na *superclasse*.

A Sobreescrita de Métodos é um conceito fundamental na programação orientada a objetos que permite a uma *subclasse* fornecer uma implementação específica de um método que já é definido em sua *superclasse*. Isso é feito utilizando a anotação **@Override** em Java, que indica que o método na *subclasse* está substituindo o método correspondente na *superclasse*. A Sobreescrita de Métodos é essencial para o **polimorfismo**, permitindo que objetos de diferentes classes sejam tratados de maneira uniforme, mas com comportamentos específicos.

Quando um método é sobrescrito, a versão do método na *subclasse* é chamada em vez da versão na *superclasse*. Isso permite que a *subclasse* adapte ou estenda o comportamento do método da *superclasse*. Por exemplo, se uma *superclasse* **Animal** define um método *emitirSom()*, uma *subclasse* **Cachorro** pode sobrescrever esse método para fornecer uma implementação específica que faz o som de um cachorro. Assim, quando o método *emitirSom()* é chamado em um objeto *Cachorro*, a implementação da *subclasse* é executada, permitindo comportamentos específicos enquanto mantém uma interface comum.

O conceito de **herança** está intimamente ligado à Sobreescrita de Métodos na programação orientada a objetos. Herança é o mecanismo pelo qual uma

⁸ Em Java, o conceito de "contrato da *superclasse*" refere-se ao conjunto de métodos e comportamentos que uma *superclasse* define e que suas *subclasses* devem ou podem implementar. Esse contrato é estabelecido através de métodos abstratos ou concretos na *superclasse*, garantindo que todas as *subclasses* compartilhem uma interface comum e comportamentos consistentes.

classe (*subclasse*) pode herdar propriedades e comportamentos de outra classe (*superclasse*). Isso permite que a subclasse reutilize o código da superclasse e, ao mesmo tempo, adicione ou modifique funcionalidades específicas.

A Sobreescrita de Métodos ocorre quando uma *subclasse* fornece uma implementação específica de um método que já foi definido na sua *superclasse*. Para que a Sobreescrita de Métodos seja possível, a subclasse deve herdar da *superclasse*. Dessa forma, a *subclasse* pode adaptar ou estender o comportamento dos métodos herdados, garantindo que a nova implementação seja chamada em vez da versão original da *superclasse*. Portanto, a Herança é um pré-requisito para a Sobreescrita de Métodos, pois é através dela que a *subclasse* obtém acesso aos métodos da *superclasse* que podem ser *sobrescritos*.

7.3 Representação de Herança em UML

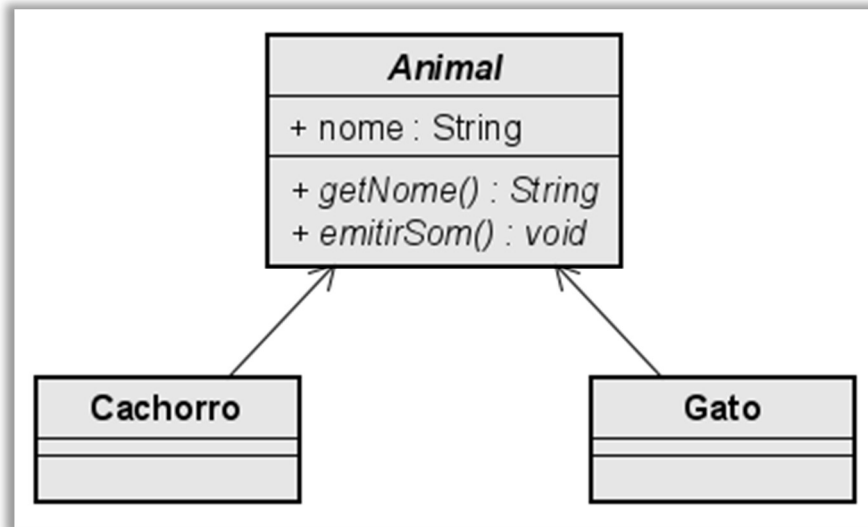


No diagrama de classes UML, o conceito de **herança** é representado por uma linha sólida conectando duas classes, com uma seta aberta apontando para a classe base (também chamada de classe "pai" ou "*superclasse*"). Esse relacionamento indica que a classe derivada (ou "filha" ou "*subclasse*") herda as características e comportamentos definidos na classe base, podendo estender ou especializar essas funcionalidades.

Por exemplo, considere a classe **Animal** como a classe base, que possui atributos e métodos genéricos como nome, idade, e *emitirSom()*. As classes **Cachorro** e **Gato** seriam *subclasses* que herdam esses atributos e métodos, mas também podem adicionar comportamentos específicos, como *latir()* no caso do **Cachorro** e *miar()* no caso do **Gato**. No diagrama UML, a classe **Animal** ficaria no topo, com uma seta aberta apontando para ela a partir de **Cachorro** e **Gato**. Esse modelo visual ajuda a compreender que tanto **Cachorro** quanto **Gato** compartilham características comuns de **Animal**, mas possuem suas próprias particularidades, destacando a ideia de reutilização de código e especialização de comportamento.

7.3.1 Exemplo de Diagrama de Classes com Herança

Imagem_7.1: Diagrama de Classes UML, representando as classes **Animal**, **Cachorro** e **Gato**.



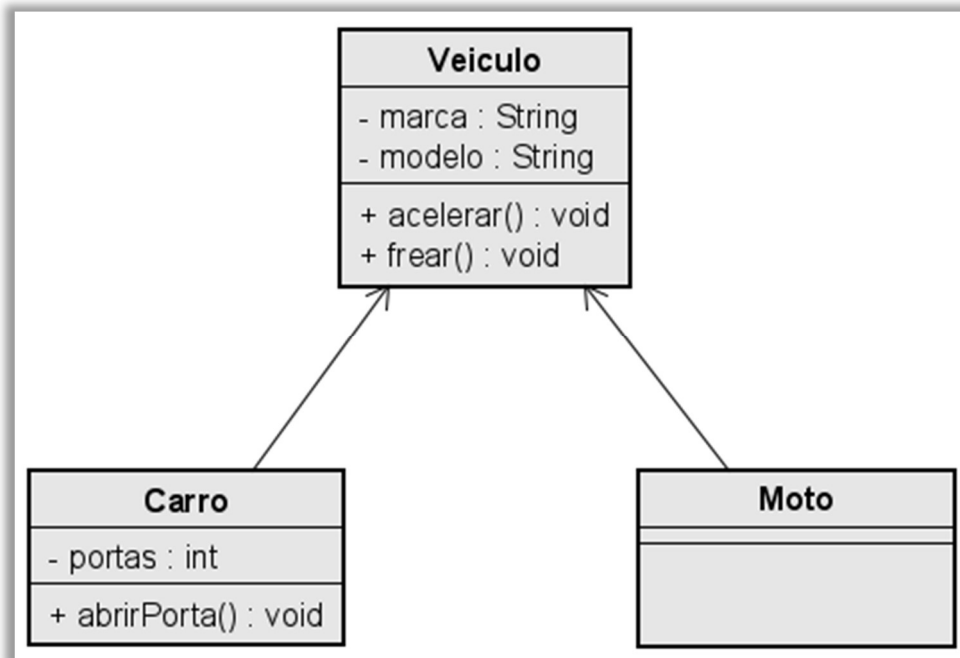
Nesse diagrama:

- **Animal** é a *superclasse*.
- **Cachorro** e **Gato** são *subclasses* de **Animal** e herdam seus atributos e métodos, podendo sobrescrevê-los conforme a necessidade.

7.3.2 Exemplo Mais Complexo

Considere uma hierarquia de veículos: **Veiculo** é uma classe geral, **Carro** e **Moto** são *subclasses* que acrescentam atributos e comportamentos específicos.

Imagem_7.2: Diagrama de Classes UML, representando as classes **Veiculo**, **Carro** e **Moto**.



- **Veiculo** contém atributos genéricos (marca, modelo) e métodos genéricos (acelerar, frear).
- **Carro** herda esses atributos e métodos, adicionando o atributo portas e um método específico *abrirPorta()*.
- **Moto** pode ter outras particularidades (como capacidadeDoTanque), que poderiam ser adicionadas.

Resumo do Capítulo



- **Herança:** Permite que classes compartilhem atributos e métodos comuns, evitando duplicação de código e facilitando a manutenção.
- **Superclasse e Subclasse:** A *superclasse* fornece características genéricas, enquanto a *subclasse* as especializa ou as estende.
- **Palavra-Chave `extends` em Java:** Utilizada para indicar que uma **classe filha** herda de uma **classe mãe**.
- **Construtor e `super()`:** A *subclasse* pode chamar o construtor da *superclasse* para garantir que seus atributos sejam devidamente inicializados.
- **Representação UML:** Uma seta sólida com ponta aberta indica a relação de herança, mostrando claramente a hierarquia de classes.

Exercícios



- **Explicando Herança:** Em suas próprias palavras, defina herança e dê um exemplo simples de uma relação de herança que você poderia encontrar no mundo real.
- **Superclasse e Subclasse:** Crie uma *superclasse* **Pessoa** com atributos nome e idade e métodos *getNome()*, *setNome()*, *getIdade()*, *setIdade()*. Em seguida, crie uma *subclasse* **Estudante** que estenda **Pessoa**, adicionando o atributo matricula e métodos *getMatricula()* e *setMatricula()*.
- **Sobrescrita de Métodos:** No exemplo do **Animal** e **Cachorro**, o método *emitirSom()* é *sobrescrito* na *subclasse*. Crie uma outra *subclasse* de **Animal**, chamada **Passaro**, que *sobrescreva* *emitirSom()* para imprimir “Piu piu!”.
- **Construtor da Superclasse:** Explique por que é importante chamar o *construtor* da *superclasse* (*super()*) no construtor da *subclasse*. Dê um exemplo em código.
- **Diagrama UML Prático:** Faça um diagrama UML simples representando uma hierarquia de classes para veículos de transporte público, onde **Veiculo** é a *superclasse*, **Onibus** e **Trem** são *subclasses* que acrescentam atributos e métodos específicos.

Conclusão



A **Herança** é um conceito poderoso, mas deve ser usada com cuidado. Nem sempre a **Herança** é a melhor solução para reaproveitar código — em alguns casos, a **composição** pode ser mais adequada. Além disso, heranças muito profundas podem tornar o código difícil de entender e manter. Ao equilibrar o uso da herança com outros princípios de design, você cria sistemas orientados a objetos mais eficientes e flexíveis.

Com estes conceitos e exercícios, você agora está familiarizado com a herança, um dos pilares da Orientação a Objetos. No próximo capítulo, exploraremos outro pilar fundamental: o **Polimorfismo**. Essa combinação — abstração, encapsulamento, herança e polimorfismo — permite criar sistemas robustos, flexíveis e elegantes.